# Are Self-Evolving Agent Skills Learning Skills?

**Anonymous submission**


## Abstract

Self-evolving agent systems revise persistent skill artifacts using execution traces and feedback. We study how their task-level utility relates to retained behavioral improvement. Across five standard SkillsBench evaluations of SkillEvolver, SPARK, and CoEvoSkills, mean paired gains over each evaluation's skill-free baseline range from 1.84 to 25.61 percentage points on 81–86 tasks. Most tasks have unchanged rewards, while the observed gains are concentrated in a subset of tasks. We complement these evaluations with lifecycle interventions on SkillRevise, SkillHone, and SkillOpt, covering fresh deployment, controlled task variation, feedback and selection pathways, artifact components, and output-locked scoring. In a targeted 24-task panel, half the tasks change success status across fresh repeats in at least one frozen condition. Controlled parent–final probes identify retained effects in Invoice and Energy, while artifact surgery localizes the Invoice effect to a specific component. Fixed-output comparisons reveal sensitivity to verifier choice. Together, the experiments connect average skill utility to the artifact, deployment, and measurement processes that produce it, and identify when an observed gain persists as reusable behavior.


## Introduction

Persistent agent skills offer an appealing route to continual improvement. Instead of updating model parameters, an agent can revise an external artifact that stores reusable instructions, scripts, references, templates, or other supporting resources. Recent systems generate, repair, select, and retain such artifacts from execution traces and evaluator feedback, often reporting higher post-evolution benchmark scores as evidence that the agent has learned a better skill (Li and Hu 2026; Liu et al. 2026b; Yang et al. 2026; Xia et al. 2026).

A higher score is not, by itself, an identified skill effect. The selected run may be unusually favorable; the optimizer may produce variable candidates; a validation gate may retain a narrow patch; repeated execution may eventually succeed without a durable behavioral change; and a verifier may reward only part of the intended behavior. An endpoint score alone cannot locate where the improvement resides.

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

We therefore ask two connected questions. When self-evolution raises the benchmark score, does the revised skill itself retain a behavioral improvement under fresh deployment and controlled task variation? Regardless of whether it does, which parts of the evolution-and-evaluation lifecycle produced the observed gain? We first quantify standard-task utility for SkillEvolver, SPARK, and CoEvoSkills on SkillsBench. We then study retention and attribution in SkillRevise, SkillHone, and SkillOpt, which represent trace-conditioned revision, history-based iterative honing, and validation-gated bounded editing. Together, these evaluations connect task-level gains with controlled studies of the mechanisms that produce them.

Our audit follows the lifecycle from feedback to revision, selection or adoption, frozen artifact deployment, and measurement. We separate historical success from fresh retention, final-skill utility from actual parent-to-final revision effects, and same-task retention from controlled transfer. We then intervene on feedback, optimizer sampling, gate provenance, artifact components, and verifier choice. A short information-integrity check bounds direct answer or oracle exposure as an alternative explanation; it is not the paper's organizing hypothesis.

The five standard-task evaluations show positive mean paired gains of 1.84–25.61 points, with most tasks receiving unchanged rewards. Revised artifacts can cause real behavioral improvements, but the demonstrated effects are often narrow. Single-run or selected gains need not survive fresh deployment. Useful traces and feedback must be converted into an effective edit, adopted into the persistent artifact, and retained behaviorally. Finally, a better score can reflect compatibility with a measurement surface rather than broader task behavior.

Our central contribution is a claim-grounded lifecycle audit: each reported gain is matched to the intervention required to support it and to the strongest conclusion that intervention permits. We make three contributions:

- **Identification framework.** We separate a reported gain from retention, behavioral scope, evolution-and-selection pathway, and measurement dependence.
- **Lifecycle audit.** We combine standard-task utility evaluations with matched interventions across three representative skill-evolution mechanisms.

- **Reporting protocol.** We specify the lineage, contrasts, and artifacts required to support each claim level, from selected gain to bounded transfer.

# Problem Formulation

## Skill Revision and Evaluation

At revision round $r$, an optimizer updates the current persistent skill artifact:

$$s_{r+1} = O(s_r, \tau_r, f_r), \tag{1}$$

where $s_r$ is the current skill, $\tau_r$ is an execution trace, and $f_r$ is feedback that may include a verifier result, a gold answer, a diagnosis, or evaluator feedback. The operator $O$ denotes the method's complete update pathway, including its native merge, validation, selection, or adoption mechanism. A generated candidate that is rejected or never adopted is not $s_{r+1}$.

The agent then executes task $x$ with the frozen skill:

$$o_r = A(x, s_r, z_r), \tag{2}$$

where $z_r$ represents model sampling, tool use, and environment state, and $o_r$ is the resulting output artifact or state change. A verifier maps the output to a score:

$$y_r = V(x, o_r). \tag{3}$$

## The Identification Question

Most evaluations observe:

$$y_{r+1} > y_r. \tag{4}$$

This inequality describes two evaluations; it does not identify what the persistent skill learned. The revised artifact may encode a retained behavioral improvement, the selected score may depend on optimizer or execution variation, or the output may better match the verifier's measurement surface. Infrastructure-invalid observations, such as a verifier that never reaches its semantic assertions, are neither successes nor failure mechanisms and must be repaired or excluded.

Accordingly, we separate two questions:

1. **Effect existence and scope:** Does the revised skill retain a behavioral improvement under fresh deployment, and how far does that effect extend under controlled task variation?
2. **Effect attribution:** Which feedback, revision, selection, artifact, execution, and measurement mechanisms produced the observed score change?

The next two sections address these questions in turn.

# Experiments: Skill Utility and Retention

## Evaluation Design and Contrasts

Our experiments combine standard-task utility evaluations, repeated deployment, and controlled mechanism probes. The standard-task study evaluates SkillEvolver (THU-AICosmos 2026), SPARK (Zhou et al. 2026b), and CoEvoSkills (Zhang et al. 2026a) against same-task skill-free baselines. The lifecycle studies analyze three systems with distinct revision mechanisms. SkillRevise converts execution traces and diagnoses into bounded repair principles (Liu et al. 2026b). SkillHone accumulates practice history and iteratively reflects on the persistent skill (Li and Hu 2026). SkillOpt produces structured bounded edits and uses a validation gate and rejected buffer before adoption (Yang et al. 2026). The following contrasts specify the task sets and comparison conditions for each study.

At the paper level, the evidence is exploratory: individual interventions use frozen inputs and predeclared gates, but the combined studies are designed to identify candidate mechanisms rather than estimate benchmark-wide prevalence. A targeted 24-task SkillsBench panel tests whether historical outcomes survive fresh deployment. The panel is stratified to expose stability failures and method disagreements; it is not a random sample. Three controlled mechanism cases then compare complete parent and final artifacts under predeclared variations. Within each contrast, we hold the executor model, harness, task instance, budget, and verifier version fixed unless one is the named intervention; exact prompts, hashes, seeds, and stopping decisions are preserved in the artifact package. The task is the primary analysis unit, while repeated runs are deployments of the same task-condition pair.

Four comparisons must remain distinct. Historical-pass to fresh-repeat comparisons test retention of a selected outcome, not advantage over a baseline. Final-to-no-skill comparisons estimate fresh utility, not the effect of revision. Only actual-parent to final comparisons identify a revision effect. Applying that comparison to controlled variants tests bounded transfer, not population-average benefit.

## Skill Utility on Standard SkillsBench Tasks

We first measure the task-level utility of each skill-evolution pipeline relative to skill-free execution. The standard-task evaluations cover SkillEvolver, SPARK, and CoEvoSkills. For each evaluation, we compare the method condition with its baseline on the same tasks and average the task-level reward differences:

$$\widehat{\Delta}_m = \frac{1}{|S_m|} \sum_{i \in S_m} \left( r_{i,m} - r_{i,\varnothing} \right), \tag{5}$$

where $S_m$ is the evaluated task set, $r_{i,m} \in [0,1]$ is the method-condition reward, and $r_{i,\varnothing}$ is its skill-free counterpart. Table 1 reports mean rewards on a 0–100 scale, paired gains in percentage points, and the numbers of improved, regressed, and tied tasks.

All five evaluations show positive mean paired gains. SkillEvolver gains 25.61 and 25.02 points in its two 81-task evaluations, while CoEvoSkills gains 10.12 points over 86 tasks. SkillEvolver's baseline mean rises by 7.12 points between evaluations, while its paired gain changes by only 0.59 points. Its two evaluations thus produce similar average gains despite shifts in absolute reward.

SPARK's two server-native evaluations cover the same 84 tasks. Rep01 increases mean reward from 17.82 to 19.66, a gain of 1.84 points; Rep02 increases it from 14.37 to 20.81, a gain of 6.45 points. The positive direction is shared across

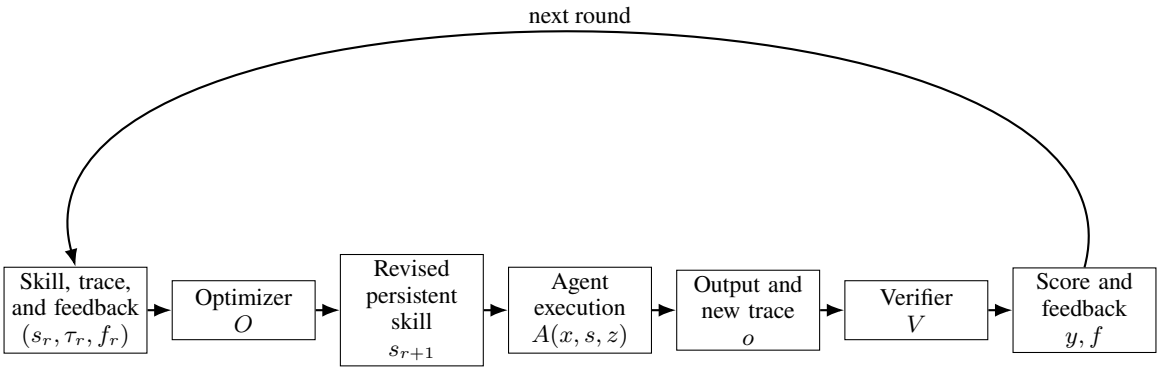



Figure 1: Minimal self-evolution lifecycle. The update operator includes any method-native validation, selection, merge, or adoption before the revised skill is retained.

| Evaluation | $n$ | No skill | Method | $\Delta$ (pp) | Improved | Regressed | Tied |
|---|---|---|---|---|---|---|---|
| SkillEvolver, first | 81 | 30.20 | 55.81 | +25.61 | 27 | 6 | 48 |
| SkillEvolver, second | 81 | 37.32 | 62.34 | +25.02 | 26 | 3 | 52 |
| SPARK, Rep01 | 84 | 17.82 | 19.66 | +1.84 | 9 | 8 | 67 |
| SPARK, Rep02 | 84 | 14.37 | 20.81 | +6.45 | 13 | 5 | 66 |
| CoEvoSkills, first | 86 | 41.72 | 51.83 | +10.12 | 17 | 9 | 60 |

Table 1: Standard-task utility relative to each evaluation's same-task skill-free baseline. Mean rewards are multiplied by 100; gains use unrounded paired differences. SPARK Rep01 and Rep02 are the two server-native evaluations. Execution settings and source selections are documented in Appendix A.

both evaluations, while the magnitude varies by 4.61 points. Each gain is calculated against the baseline from its own evaluation.

Positive averages coexist with uneven task-level outcomes. Across the five evaluations, 9–27 tasks improve, 3–9 regress, and 48–67 tie. Unchanged rewards account for most tasks in every evaluation, so the aggregate gains are concentrated in a subset of tasks. This pattern motivates examining which improvements persist when skills are redeployed or task requirements change.

Appendix A records execution settings, task coverage, and recovery accounting for each evaluation.

## Retention under Fresh Deployment

This experiment tests whether historically selected outcomes survive when the final artifact is frozen and redeployed. Holding task, skill, and condition fixed does not make execution deterministic. Across the 24-task panel, 12 tasks change success status between their two repeats in at least one condition. Aggregate fresh success is 14/48 without a skill, 16/48 with the SkillRevise final artifact, and 12/48 with the SkillHone final artifact. Both matched-lift intervals cross zero (Table 2).

These results establish an identification failure in the common reporting practice: cached or selected success cannot be read as stable deployment capability. They do not establish that either revision method is ineffective. The panel is purposive, and it lacks each method's actual parent condition. Its proper conclusion is narrower: on this sample, neither final-versus-no-skill comparison establishes a clear average fresh-deployment advantage, and fresh repeats are necessary before a selected score is called retained.

## Revision Effects under Controlled Variations

These mechanism probes test whether the actual parent-to-final revision effect persists under predeclared task changes. In Invoice, the complete final skill repairs all 12 matched deployments and retains the effect on nine deployments drawn from three disjoint corpora that preserve the contract while changing the documents. This is strong evidence of a stable, same-contract revision effect, but not of broad cross-contract generalization.

The Energy case provides weaker but directionally consistent evidence: the complete final artifact improves the original setting and a 5% parameter variant in five of six deployments, compared with one of six for the parent. The sample is too small to support a prevalence or generality claim. In SkillOpt SearchQA, the final artifact improves independent held-out accuracy from 9/24 to 17/24, yet fails the predeclared stability threshold. It therefore supports local held-out benefit rather than stable transferable procedure learning.

Taken together, revised skill artifacts can produce retained behavioral effects, but a selected benchmark pass alone does not identify such an effect, and the effects established so far remain narrow rather than broadly generalizable.

| Panel and contrast | Main result | Supported inference and boundary |
|---|---|---|
| **Panel A: Retention under fresh deployment** | | |
| Historical pass → fresh repeats | 12/24 tasks flip within at least one frozen condition; selected-outcome retention is 47.1% for SkillRevise and 55.0% for SkillHone | A selected pass is not a deployment probability; denominators differ and are not a method ranking. |
| SkillRevise final → no-skill | 16/48 vs. 14/48; $+4.2$ pp, 95% CI $[-10.4, 20.8]$ | No clear fresh-utility advantage on this targeted panel; not a parent-to-final revision estimate. |
| SkillHone final → no-skill | 12/48 vs. 14/48; $-4.2$ pp, 95% CI $[-20.8, 14.6]$ | No clear fresh-utility advantage on this targeted panel; not a parent-to-final revision estimate. |
| **Panel B: Revision effects under controlled variations** | | |
| Invoice parent → final | $0/12 \rightarrow 12/12$; three new same-contract corpora: $0/9 \rightarrow 9/9$ | Stable but narrow same-contract transfer. |
| Energy parent → final | Original task plus 5% parameter variant: $1/6 \rightarrow 5/6$ | Directional parameter transfer in a small stochastic case. |
| SkillOpt SearchQA parent → final | Independent held-out $9/24 \rightarrow 17/24$; stable-transfer gate fails | Local held-out gain, not stable general procedure learning. |

Table 2: Evidence for retained skill effects. Panel A contains 24 targeted tasks, three conditions, and two fresh deployments per condition (144/144 behavior-valid runs). Different historical-retention denominators preclude horizontal method ranking. The no-skill condition calibrates execution and selection variability; final-versus-no-skill does not identify revision effects.

| Task | Variant | None | SE | SP | CE |
|---|---|---|---|---|---|
| Earthquake plate | P3 | 0 | 1 | 1 | 1 |
| Exoplanet period | P1 | 0 | 1 | 0 | 0 |
| Exoplanet period | P2 | 1 | 0 | 0 | 0 |
| Exoplanet period | P3 | 0 | 1 | 0 | 0 |
| Financial modeling | P2 | 1 | 1 | 0 | 1 |

Table 3: Illustrative historical variant rewards on the original 0–1 scale. SE: SkillEvolver; SP: SPARK; CE: CoEvoSkills. These examples show higher and lower rewards relative to no skill; the full inventory reports all 222 variants.

### SkillsBench Task Variations

We examine how recorded skill utility varies across task changes using 222 SkillsBench variants and four conditions: no skill, SkillEvolver, SPARK, and CoEvoSkills. The collection distinguishes multi-axis perturbations (P1), structural changes (P2), and coupled changes to task requirements (P3). Table 3 presents selected historical outcomes that illustrate differences across variants and methods.

The direction of the observed difference changes within a task family. On Exoplanet P1 and P3, SkillEvolver receives reward 1 while the skill-free condition receives 0; on P2, the skill-free condition receives 1 and all three method conditions receive 0. Other examples distinguish shared success from method-specific failure: all three methods receive 1 on Earthquake plate P3, compared with 0 without a skill, whereas SPARK alone receives 0 on Financial modeling P2. These cases identify task changes under which the observed utility of a skill differs, motivating the parent–final interventions used to investigate retained behavior.

The full inventory reports 838 historical task–condition entries and 48 recovery entries separately by execution protocol, with two further entries excluded for unavailable transcription service. Appendix A documents the exclusions, five historical outcome conflicts, and three file-reading reevaluations.

## Attributing Observed Skill Gains

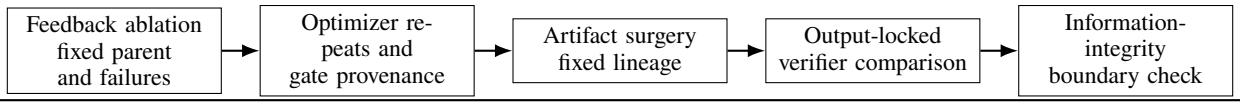
### Lifecycle Interventions

The experiments in this section intervene at different lifecycle stages. They do not estimate one common effect size. Each intervention changes one object while holding its relevant lineage fixed: feedback-view studies hold the parent and failures fixed; optimizer-repeat studies hold the input evidence fixed; gate audits preserve accepted and rejected candidates; artifact surgery holds the surrounding artifact fixed; and verifier counterfactuals hold the output fixed.

Figure 2 makes the current coverage and its gaps explicit. The shared audit logic is that feedback access, candidate revision, validation acceptance, adoption into the persistent artifact, and retained behavioral improvement are distinct events.

### Revision and Selection Pathways

These interventions test whether the observed endpoint depends on the feedback view, optimizer sample, or selection gate. In SkillRevise, the complete three-revision pipeline succeeds on 53/86 tasks, compared with 28/86 when diagnosis is removed. Task-level discordance is 26 gains versus one loss, and 2,326 of 3,648 revision entries explicitly use verifier evidence. This demonstrates that diagnosis changes the selected endpoint and that the revision process is strongly evaluator-facing. It does not, by itself, establish which changes are retained under fresh deployment.




| Lifecycle stage | SkillRevise | SkillHone | SkillOpt |
|---|---|---|---|
| Feedback and revision | Diagnosis ablation; revision traces | Feedback-view ablation | Feedback branches; optimizer repeats |
| Selection and adoption | Revision history | Iterative honing history | Native validation gate; accepted and rejected candidates |
| Artifact mediation | Not yet matched at the same strength | Invoice and Energy surgery | SearchQA component surgery |
| Measurement | No output-locked comparison yet | Radar verifier swap | Spreadsheet dual scoring |
| Information integrity | Artifact forensic | Artifact forensic | Wrong-metadata and oracle controls |

Figure 2: Lifecycle attribution design and current evidence coverage. The upper panel locates each intervention; the lower panel records method coverage. Populated cells do not imply equivalent strength, and missing cells bound cross-method claims.

SkillHone separates access to feedback from stable benefit. Detailed sanitized, binary, and redacted-tail feedback obtain first-deployment success on 4/12, 4/12, and 5/12 cases, while four of 18 repeated cells flip. Feedback can trigger revisions, but these data do not support a stable advantage for one feedback view.

SkillOpt further separates candidate production from attribution and adoption. In matched SearchQA optimizer repeats, raw-gold and outcome-only views each produce four applied edits. Yet the preregistered between-condition effect size is only $D = .044 < .20$, despite weak lexical decodability, so a single patch cannot be cleanly attributed to feedback view. Across outcome-only and trajectory-only branches, all 6/6 candidates change and 3/6 pass the gate, but every acceptance is driven by the same Blackwater validation item. In OfficeQA, a selected one-item gain disappears in fresh matched deployment. Thus edit production, gate acceptance, and retained improvement cannot be collapsed into one event.

## Artifact-Level Mediation

This experiment tests whether a feedback-derived component, rather than unrelated edits in the full artifact, causes the behavior after adoption. Let $P$ denote the complete parent, $F$ the complete final artifact, $P + B$ the parent with component $B$, and $F - B$ the final artifact with that component removed.

| Case | Result | Maximum claim |
|---|---|---|
| Invoice | $P = 0/12$, $P + B = 12/12$, $F - B = 0/12$, $F = 12/12$ | $B$ is necessary and sufficient for the narrow same-contract effect. |
| Energy | Local mediation in the original task; necessity and sufficiency do not survive the variant | Local component effect, not transferable mediation. |
| SearchQA | 9 repairs, 0 regressions; stable success 7/12, below the strong gate | Directional mediation on part of the held-out set. |

Table 4: Artifact-level mediation evidence.

These interventions establish that an adopted artifact component can cause behavior. They do not show that the same mechanism is prevalent across methods. In particular, SkillRevise lacks a matched surgery of equal strength, and the current cases should be read as existence and boundary evidence.

## Measurement Dependence under Fixed Outputs

This counterfactual tests whether a verdict changes when the output is fixed and only the measurement changes. In the Radar family, all 12 reconstructed outputs reproduce their historical original-verifier outcomes. Of six original passes, only one remains a pass under a reference-calibrated strict verifier; the five valid flips arise from specification-implied checks. The same behavior can therefore receive different valid verdicts because the measurement surfaces differ.

Measurement dependence can also operate in the other direction. For identical Spreadsheet workbook outputs, the reported value-based evaluator scores the complete SkillOpt artifact at 10/12, whereas joint value-and-formula recalculation scores it at 8/12. Representation, cache materialization, or formula inspection can therefore change what the evalua-

tor observes. This dual scoring is not identical in design to the Radar output-locked swap, but it reinforces the narrower point that score is a property of output plus measurement, not behavior alone.

Neither result proves semantic failure, exploitation, or benchmark-wide prevalence. To identify evolution-induced verifier dependence, one would additionally need matched no-skill, parent, and final outputs under both original and calibrated verifiers, and test whether the parent-to-final verifier gap increases.

### Information-Integrity Boundary

This supporting check tests whether restricted answer or oracle information enters and controls the artifact. High-confidence direct-answer or oracle matches are absent from 86 SkillRevise and 85 SkillHone natural artifacts. SkillOpt conditions containing the feedback-derived answer-surface section follow incorrect metadata in 0/15 probes. In contrast, an unsafe hidden-oracle positive control changes probe success from 0/5 to 5/5 and leaves private-oracle or gold markers.

These checks show that the audit can detect direct exposure when it is deliberately introduced, while the natural artifacts do not support direct copying as a universal explanation. They do not prove the absence of implicit answer-following policies or all forms of leakage.

Observed score gains can arise at multiple lifecycle stages: feedback shapes candidate revisions, selection can amplify narrow improvements, adopted artifact components can mediate real behavior, and valid verifiers can disagree on fixed outputs. These mechanisms must be identified separately before a score gain is interpreted as skill learning.

## From Reported Gains to Auditable Claims

### Evidence Requirements and Claim Boundaries

The audit is not a scalar ladder from *safe* to *unsafe*. Its steps answer independent questions and impose distinct claim boundaries.

| Audit step | Required test → maximum supported claim |
|---|---|
| Validity | Execution and verifier status plus assertion coverage → eligible observation |
| Retention | Frozen actual-parent/final fresh repeats → retained artifact effect |
| Behavioral scope | Entity, parameter, input, contract, or family variants → bounded controlled transfer |
| Evolution pathway | Feedback ablation, optimizer repeats, and gate provenance → pathway dependence or selection mediation |
| Measurement | Output-locked calibrated verifier comparison → measurement dependence |
| Information integrity | Provenance, wrong-metadata, and positive control → exploitation or confirmed leakage, when supported |

Table 5: Evidence requirements and claim boundaries. The rows answer complementary questions rather than forming a scalar risk ladder.

A selected gain is weaker than a retained artifact effect; a retained effect on one instance is weaker than controlled transfer. Evolution-pathway and measurement claims remain meaningful even when retention is not established. Conversely, verifier-aware language or evaluator dependence must not be relabeled as leakage without restricted-information provenance and behavioral evidence.

### Lifecycle Reporting Protocol

An auditable self-evolution report should preserve the following minimum evidence:

- **Before optimization:** parent artifact, task and split identity, failure pool, environment, and verifier version.
- **During revision:** optimizer-visible traces and feedback, candidate patches, optimizer samples, prompts, and usage.
- **At selection and adoption:** accepted and rejected artifacts, gate items, per-item decisions, and exact adopted artifact.
- **At deployment:** frozen artifact, actual-parent and no-skill conditions, fresh repeats, controlled variants, and output bytes.
- **At scoring:** verifier logs, assertion coverage, reference calibration, and any alternative verifier applied to the same output.
- **For cost:** separate optimizer, executor, verifier, and invalid-attempt ledgers.

The complete artifact appendix should additionally record hashes, commands, seeds, environment revisions, timeouts, and stopping decisions. This protocol turns a post-evolution score into an auditable lineage and limits the paper's language to the strongest matched contrast actually performed.

## Related Work

### Skill Evolution and Empirical Evaluation

Agent-skill research spans generation, revision, selection, retrieval, organization, and reinforcement learning. SkillsBench and SWE-Skills-Bench compare skill and no-skill utility across diverse or software-engineering tasks (Li et al. 2026; Han et al. 2026). SkillLearnBench evaluates continual skill generation (Zhong et al. 2026), while SkillRevise, SkillHone, SkillOpt, SkillRL, and CoEvoSkills propose different revision or learning loops (Liu et al. 2026b; Li and Hu 2026; Yang et al. 2026; Xia et al. 2026; Zhang et al. 2026a). Recent empirical work broadens the object of study: From Raw Experience to Skill Consumption separates experience generation, extraction, and consumption (Huang et al. 2026); How Well Do Agentic Skills Work in the Wild studies realistic skill retrieval (Liu et al. 2026a); SkillCoach evaluates process-level skill use (Zhu et al. 2026); and SkillJuror isolates the effect of skill organization (Chen et al. 2026).

These studies show that outcome-only evaluation is already recognized as incomplete. Our focus is narrower and complementary: when a persistent skill is revised and the post-revision score rises, what evidence identifies a retained artifact effect rather than a selected or measurement-dependent endpoint? We therefore organize evidence by lifecycle contrast rather than by method score.

### Evaluation and Attribution of Agent Improvement

Agent evaluation increasingly examines process quality, sequential adaptation, verifier reliability, and counterfactual behavior. Counterfactual Trace Auditing measures how a provided skill changes an agent trajectory (Zhou et al. 2026a). SEA-Eval evaluates self-evolving agents beyond isolated episodes (Jiang et al. 2026). Co-evolving verifier work and grader audits study interactions between the policy and its measurement signal (Zhang et al. 2026a,b). More broadly, reward hacking and test-set contamination research demonstrates that performance on a proxy or exposed test need not imply the intended capability (Amodei et al. 2016; Oren et al. 2023).

We combine these concerns within the lifecycle of persistent skill revision. The central distinction is between retention, behavioral scope, evolution-and-selection pathway, and output-level measurement dependence. Information-integrity checks play a supporting role: they bound direct answer or oracle exposure, but do not convert every verifier-aware adaptation into a leakage claim.

## Limitations

Standard-task evaluations differ in task coverage, execution backends, and time budgets; each reported gain uses its own same-task baseline. The lifecycle studies focus on individual mechanisms. The 24-task retention panel is purposive and contains only two fresh deployments per condition; it exposes instability but cannot estimate deployment rates. The controlled Invoice, Energy, SearchQA, Radar, Spreadsheet, and OfficeQA probes use small, heterogeneous task families rather than a fully crossed method-by-intervention design. SkillRevise, for example, lacks matched artifact surgery and output-locked measurement evidence of the same strength, while the strongest verifier swap is concentrated in the Radar family. Variant observations are reported by task and condition, with execution protocols and conflicting historical outcomes retained in the inventory.

Several conclusions are also bounded by observability. Output-locked comparisons identify measurement dependence but do not determine which verifier best captures intended behavior without an external specification. Artifact forensics and wrong-metadata controls can rule out simple direct exposure, but not implicit answer-following policies or every form of contamination. Finally, our scope is frozen-agent, feedback-driven systems with persistent external artifacts; parameter-updating self-evolution, multimodal skills, and online environments may exhibit different lifecycle dynamics. Confirmatory evidence will require a shared sampling plan that fully crosses actual parent and final artifacts, fresh repeats, controlled variants, and calibrated verifiers.

## Conclusion

Across five standard SkillsBench evaluations, SkillEvolver, SPARK, and CoEvoSkills yield positive mean paired gains of 1.84–25.61 points over their respective skill-free baselines. These gains are concentrated in a subset of tasks, and the variant examples show changes in the direction of the observed difference within a task family. The lifecycle studies identify retained artifact effects, deployment instability, and transfer that varies across task changes. Feedback, optimizer variation, selection, artifact content, execution, and verifier choice shape these outcomes.

Our lifecycle audit links observed gains to the interventions that identify their source. Evidence for reusable skill learning comes from frozen parent–final improvement that persists under fresh deployment and controlled task variation. Combining utility evaluations with these interventions connects benchmark performance to the behavior carried by the skill artifact.

## References


Amodei, D.; Olah, C.; Steinhardt, J.; Christiano, P.; Schulman, J.; and Mané, D. 2016. Concrete Problems in AI Safety. arXiv:1606.06565.

Chen, Z.; Guo, Z.; Huang, B.; Lu, B.; Lin, J.; Zhou, Y.; and Zhang, W. 2026. SkillJuror: Measuring How Agent Skill Organization Changes Runtime Behavior. arXiv:2606.11543.

Han, T.; Zhang, Y.; Song, W.; Fang, C.; Chen, Z.; Sun, Y.; and Hu, L. 2026. SWE-Skills-Bench: Do Agent Skills Actually Help in Real-World Software Engineering? arXiv:2603.15401.

Huang, Z.; Xu, J.; Yang, Y.; Gong, Z.; Yang, Q.; Tian, M.; Wang, X.; Lv, C.; Gao, X.; Dai, Q.; Liu, B.; Qiu, K.; Yang, X.; Chen, D.; Zheng, X.; and Luo, C. 2026. From Raw Experience to Skill Consumption: A Systematic Study of Model-Generated Agent Skills. arXiv:2605.23899.

Jiang, S.; Ma, L.; Hong, Z.; Wang, K.; Lu, Z.; Wang, T.; Chen, S.; Zhang, J.; Pan, T.; Li, W.; Liang, J.; and Xiao, Y. 2026. SEA-Eval: A Benchmark for Evaluating Self-Evolving Agents Beyond Episodic Assessment. arXiv:2604.08988.

Li, X.; Liu, Y.; Chen, W.; You, B.; Di, Z.; He, Y.; Zheng, S.; Choe, K. W.; Sun, J.; Wang, S.; Tao, C.; Li, B.; Zhao, X.; Geng, H.; Wu, X.; Zhou, J.; Chen, X.; Xing, H.; Li, Y.; Zeng, Q.; Wang, D.; Wang, Y.; Chaim, R. B.; Jiang, P.; Shen, H.; Kong, L.; Liu, X.; Wang, R.; Liu, X.; Li, J.; Lan, X.; Lin, Y.; Ye, W.; He, J.; Li, S.; Zhang, Y.; Gao, Y.; Li, Y.; Ma, Z.; Jing, L.; Wang, T.; Li, K.; Xue, Y.; Lyu, H.; He, Y.; Tian, Y.; Wu, S.; Wang, B.; Gao, Y.; Chen, B.; Liu, L.; Cheng, S.; Bao, J.; Tong, S.; Xu, S.; Zhuo, T. Y.; Ye, T.; Qi, Q.; Li, M.; Liao, L.; Tan, Z.; Shi, C.; Tang, X.; Tankasala, S.; Yuan, B.; Qian, Y.; Tu, J.; Wang, C.; Sun, Y.; Wang, W.; Taylor, A.; Yang, Z.; Guan, C.; Dong, Z.; Zhang, X.; Dillmann, S.; chung Lee, H.; and Song, D. 2026. SkillsBench: Benchmarking How Well Agent Skills Work Across Diverse Tasks. arXiv:2602.12670.

Li, Z.; and Hu, Y. 2026. SkillHone: A Harness for Continual Agent Skill Evolution Through Persistent Decision History. arXiv:2606.08671.

Liu, Y.; Ji, J.; An, L.; Jaakkola, T.; Zhang, Y.; and Chang, S. 2026a. How Well Do Agentic Skills Work in the Wild: Benchmarking LLM Skill Usage in Realistic Settings. arXiv:2604.04323.

Liu, Y.; Su, Z.; Xie, L.; Zhang, Y.; Zong, Q.; Guo, J.; Xie, Z.; Ji, Y.; Yim, Y.; Luo, H.; Ren, X.; Ruan, C.; Li,

H.; and Song, Y. 2026b. SkillRevise: Improving LLM-Authored Agent Skills via Trace-Conditioned Skill Revision. arXiv:2606.01139.

Oren, Y.; Meister, N.; Chatterji, N.; Ladhak, F.; and Hashimoto, T. B. 2023. Proving Test Set Contamination in Black Box Language Models. arXiv:2310.17623.

THU-AICosmos. 2026. SkillEvolver. Software repository.

Xia, P.; Chen, J.; Wang, H.; Liu, J.; Zeng, K.; Wang, Y.; Han, S.; Zhou, Y.; Zhao, X.; Chen, H.; Zheng, Z.; Xie, C.; and Yao, H. 2026. SkillRL: Evolving Agents via Recursive Skill-Augmented Reinforcement Learning. arXiv:2602.08234.

Yang, Y.; Gong, Z.; Huang, W.; Yang, Q.; Zhou, Z.; Huang, Z.; Li, Y.; Gao, X.; Dai, Q.; Liu, B.; Qiu, K.; Yang, Y.; Chen, D.; Yang, X.; and Luo, C. 2026. SkillOpt: Executive Strategy for Self-Evolving Agent Skills. arXiv:2605.23904.

Zhang, H.; Fan, S.; Zou, H. P.; Chen, Y.; Wang, Z.; Zhou, J.; Li, C.; Huang, W.-C.; Yao, Y.; Zheng, K.; Liu, X.; Li, X.; and Yu, P. S. 2026a. CoEvoSkills: Self-Evolving Agent Skills via Co-Evolutionary Verification. arXiv:2604.01687.

Zhang, X.; Wang, G.; Cui, Y.; Li, Z.; Qiu, W.; Zhu, B.; and He, P. 2026b. Who Grades the Grader? Co-Evolving Evaluation Metrics and Skills for Self-Improving LLM Agents. arXiv:2607.12790.

Zhong, S.; Lu, Y.; Ning, J.; Wan, Y.; Feng, L.; Ao, Y.; Ribeiro, L. F. R.; Dreyer, M.; Ammirati, S.; and Xiong, C. 2026. SkillLearnBench: Benchmarking Continual Learning Methods for Agent Skill Generation on Real-World Tasks. arXiv:2604.20087.

Zhou, X.; Liu, J.; Li, L.; Rossi, R. A.; and Hu, X. 2026a. Counterfactual Trace Auditing of LLM Agent Skills. arXiv:2605.11946.

Zhou, Y.; Dong, Z.; Wang, Z.; Jin, C.; Zhao, S.; Guo, B.; Gu, D.; Zhang, L.; Zhou, M.; and Metaxas, D. N. 2026b. Evidence Over Plans: Online Trajectory Verification for Skill Distillation. arXiv:2605.09192.

Zhu, J.; Mao, K.; Guo, Y.; He, D.; Xu, S.; Gu, S.; and Yue, Y. 2026. SkillCoach: Self-Evolving Rubrics for Evaluating and Enhancing Agentic Skill-Use. arXiv:2607.01874.

## Appendix A. SkillsBench Evaluation Records

### Standard-task accounting

The first SkillEvolver evaluation combines an original 76-task summary with five completed recovery evaluations. The second 81-task evaluation combines fresh paired records and task-specific engineering recoveries; its records identify source runs, skill trees, checkpoints, and selected result paths. CoEvoSkills' first evaluation combines an 85-task audit with a recovered MHC task whose baseline and evolved rewards are both zero; scheduling-email-assistant is excluded because its verifier requires unavailable Gmail OAuth access.

SPARK Rep01 and Rep02 use the two server-native final paired summaries on the same 84-task set, excluding fix-druid-loophole-cve and syzkaller-ppdev-syzlang. Both request GPT-5.5 with medium reasoning through OpenLux and use a task timeout multiplier of 0.5. Rep01 has 43 generated skills and 41 no-skill outcomes; Rep02 has 41 generated skills and 43 no-skill outcomes. The selected reward files and result hashes agree with both summaries across all 336 condition-level evaluations. Rep01 includes recorded budget and process-cleanup repairs; its final audit preserves the selected trial paths. The earlier local subscription run copied to the server is archived and excluded from the reported SPARK results. SkillEvolver and the reported CoEvoSkills evaluation use the subscription backend.

The second CoEvoSkills evaluation has paired records for 85 unique tasks across its main run and recovery runs. Ten tasks have multiple valid records, including four with conflicting rewards: gravitational-wave-detection, lean4-proof, earthquake-phase-association, and offer-letter-generator. This evaluation remains outside the main results table pending a final manifest recording each selected result path, content hash, and selection rationale independent of reward. Evolution-artifact source resolution is recorded separately from evaluation-result selection.

### Variant scope and historical conflicts

The original collection has 228 variants. Six variants from the parallel-TFIDF and MHC families are excluded for unavailable execution resources or evolution provenance, leaving 222 variants. Two PG-audiobook P3 method cells are subsequently excluded for unavailable transcription service, giving 886 numerical-result identities. The 838 historical identities comprise 819 literal-valid entries and 19 budget-terminal or status-alias entries. Forty-eight identities have separately recorded recovery evaluations. Five historical identities retain both 0 and 1 outcomes (Table A1); their entries remain marked as conflicts in the per-variant inventory.

Table 3 selects examples with unambiguous historical rewards to illustrate changes in the direction of the observed difference and variation across methods. All 20 displayed rewards were checked against the original result files. Full task identifiers, source paths, and content hashes accompany the table in the evidence package. Historical entries and recoveries retain their own environment, skill-delivery, and grading versions; the collection is reported descriptively at the task–condition level.

| Variant | Condition | Rewards |
|---|---|---|
| Energy-market-pricing P1 | No skill | 0, 1 |
| Energy-market-pricing P3 | SPARK | 0, 1 |
| Flink-query P2 | SPARK | 0, 1 |
| Organize-messy-files P1 | SkillEvolver | 0, 1 |
| Organize-messy-files P1 | SPARK | 0, 1 |

Table A1: Historical outcome conflicts retained in the variant inventory.

### File-reading evaluations

Three recovery cells have both an earlier inline-Skill evaluation and a later file-reading evaluation (Table A2). AgentOps retains its environment-v2 configuration. P1 completes with reward zero. P2 reuses the stopped model artifact and connects the existing typed build-verdict classifier to the clean native grader; the evaluation completes in 172.96 seconds within the 240-second budget. Both tasks fail the timestamp assertion in all five Python environments.

The first Speaker attempt misses line 121 of its 122-line Skill and has a null reward. The subsequent prompt specifies complete chunked reads of the frozen file. The model run completes in approximately 564 seconds; native grading takes 268.83 seconds within the 2,700-second budget. Seven of 11 checks pass. The four failed quality checks yield DER 136.29%, JER 93.99%, WER 30.19%, and CER 30.21%, against thresholds of 20%, 25%, 20%, and 25%, respectively. The final reward is zero. The original tests and Skill bytes are retained, with the prompt and scoring-entry revisions recorded separately.

| Variant | Method | Inline | File read |
|---|---|---|---|
| AgentOps P1 | SkillEvolver | 0 | 0 |
| AgentOps P2 | SPARK | 1 | 0 |
| Speaker P3 | CoEvoSkills | 0 | 0 |

Table A2: Earlier inline and subsequent file-reading results, with execution and repair versions retained in the accompanying records.

### Evidence provenance

The accompanying evidence map records source paths, SHA-256 digests, task-level selections, and the coverage-ledger chain. Standard-task sources are the reconciled SkillEvolver first-81 report, its second-81 paired audit, the two server-native SPARK 84-task summaries, and the CoEvoSkills union87-minus-google-auth report. The second-run audit also preserves the CoEvoSkills coverage and conflicting attempts. Variant sources include the final-r5 coverage ledger and its parents, the historical attempt inventory, and the file-read-rerun-r1 and file-read-repair-r2 records. The per-variant paper view selects the three final file-reading records while retaining earlier outcomes and failed attempts in the evidence package.